



INTRODUCCIÓN AL DISEÑO LÓGICO

TRABAJO INTEGRADOR

Bernardo Etcheto 03775/3

Matias Lopez 02670/0

Joaquin Labarta 03207/3

Emiliano Paoloni 03658/8

Facultad de Ingeniería - UNLP

Índice

1. Introducción.....	1
2. Justificación del Diseño y Enfoque Adoptado.....	1
3. Máquina de Estados.....	2
3.1 Estados Definidos.....	2
3.2 Diagrama de Estados.....	2
3.3 Descripción de los estados.....	3
4. Funciones de Salida.....	4
5. Implementación en Quartus.....	5
5.1. Máquina de Estados.....	5
5.2. Sumador de 12 bits con Signo.....	6
5.2.1. Módulo sumadorCompleto.....	6
5.2.2. Módulo Sumador de 12 bits (sumador12Bits).....	7
5.2.3. Modulo Inversor.....	8
5.2.4. Módulo sumador12BitsPosYneg.....	9
5.3. Registros.....	9
5.4. Bus DATA[11..0].....	10
5.5. Contador.....	12
5.6. Implementación del Flag FIN.....	13
5.7. Implementación del Flag PAUSA.....	14
5.8. Implementación del Flag ERROR.....	15
6. Simulación Funcional con Herramienta WaveForm.....	16
6.1. Caso 1:.....	16
6.2. Caso 2:.....	17
6.3. Caso 3:.....	18
6.4. Caso 4:.....	19
6.5. Caso 5:.....	20

1. Introducción

El sistema fue pensado para realizar un conteo ascendente o descendente entre dos valores cargados externamente. Además, permite definir el paso de incremento o decremento por ciclo mediante una entrada de 3 bits (b0, b1 y b2). La gestión de los valores inicial, final y actual del contador se realiza a través de una interfaz de datos bidireccional de 12 bits (DATA[11..0]).

El funcionamiento general se controla mediante señales de entrada como LOAD_I, LOAD_F, START, UP/DOWN, RECARGAR y CONTINUAR, mientras que señales de salida como ERROR, PAUSA y FIN indican el estado del proceso.

Si bien se evaluó inicialmente el uso de múltiples máquinas de estados, se optó por una única máquina de tipo Moore. Esta decisión se basó en la simplicidad para la sincronización, sin dejar de garantizar un control efectivo del sistema.

2. Justificación del Diseño y Enfoque Adoptado

El sistema se organiza en una serie de estados que permiten cargar las configuraciones necesarias, ejecutar el proceso de conteo y responder ante situaciones de finalización o error.

Antes de iniciar el conteo, se verifica la coherencia entre los valores inicial y final, teniendo en cuenta el modo de operación (UP/DOWN). Esta validación se realiza luego de cargar el valor final. En caso de detectar una configuración inconsistente (por ejemplo, valor inicial mayor al final en modo ascendente), se activa la señal ERROR y el sistema transiciona al estado de error. Si los datos son válidos, el sistema queda a la espera de la señal START, que habilita el pasaje al estado de conteo activo.

Esta estrategia permite prevenir errores de ejecución desde el inicio, reducir la cantidad de transiciones innecesarias y mantener un número acotado de estados.

Todo el sistema fue modelado en Quartus, empleando diagramas de bloques, lógica combinacional, registros y comparadores. Las funciones de control asociadas a la máquina de estados fueron obtenidas mediante la simplificación lógica con mapas de Karnaugh.

3. Máquina de Estados

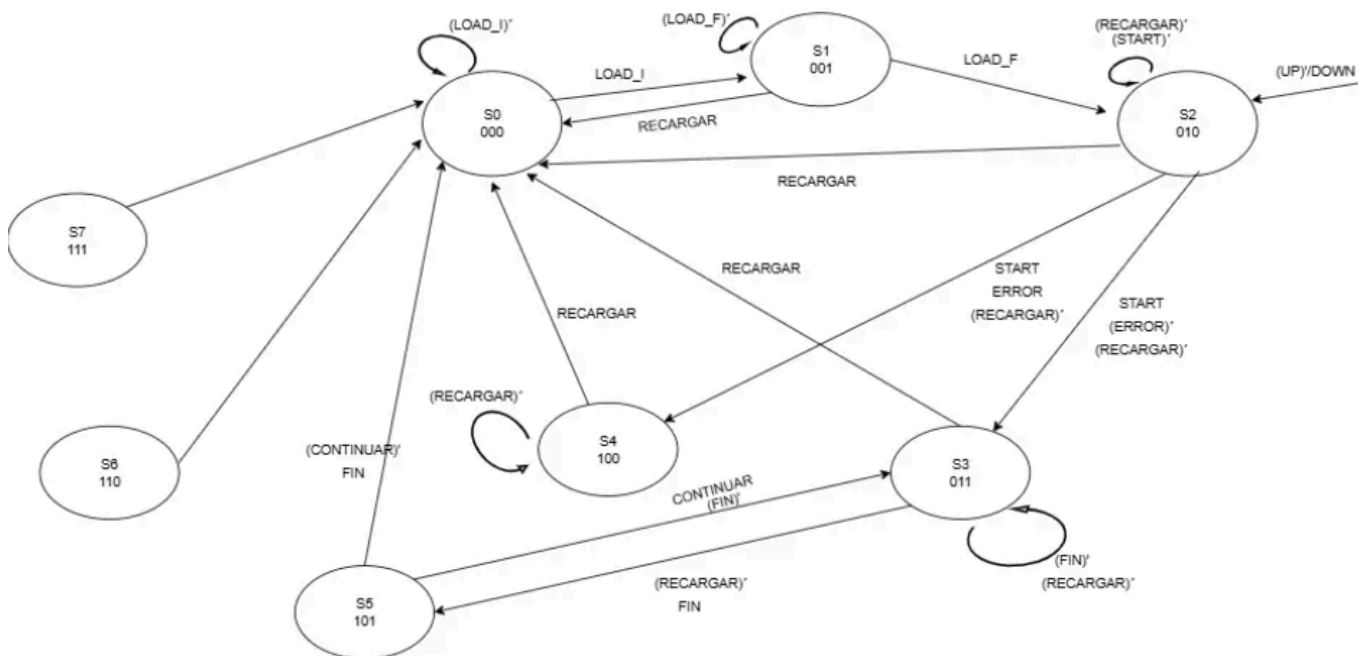
3.1 Estados Definidos

El sistema se modeló como una máquina de Moore de 3 bits (Q2 Q1 Q0), con seis estados funcionales que controlan la carga, validación, conteo, finalización y manejo de errores.

Estado	Función	Q2	Q1	Q0
S0	Inicial / Espera	0	0	0
S1	Carga valor inicial (LOAD_I)	0	0	1
S2	Carga valor final + validación	0	1	0
S3	Conteo activo	0	1	1
S4	Error	1	0	0
S5	Fin alcanzado (FIN = 1)	1	0	1
S6	Estado incoherente o no valido	1	1	0
S7	Estado incoherente o no valido	1	1	1

3.2 Diagrama de Estados

Se presenta el diagrama general de la máquina, que ilustra gráficamente cómo se realiza el paso entre estados en función de las señales activas (LOAD_I, LOAD_F, START, FIN, ERROR, etc.).



3.3 Descripción de los estados

Para facilitar la comprensión del funcionamiento del sistema, se detallan los estados definidos en la máquina de Moore:

S0: Estado inicial. Se espera a que llegue la señal *LOAD_I* para cargar el valor que se encuentra en el *bus* en el registro *valor_inicial*; sólo se copiará en el registro cuando llegue esta señal.

S1: En este estado ya se cargo el valor inicial en el registro; ahora debe suceder lo mismo con el valor final, esperando a la señal *LOAD_F* únicamente y no dándole importancia a las demás señales. La única señal que me puede llegar a importar es la de *RECARGAR*, que me vuelve al estado anterior de cargar el valor inicial nuevamente.

S2: Estado donde ya están cargados los valores inicial y final en sus respectivos registros y esperamos a que llegue la señal *START* para empezar a contar. Antes de eso, se debe configurar la señal *UP/DOWN* que me indique qué desplazamiento hará el sumador. En caso de que los valores inicial y final sean incoherentes con el desplazamiento elegido, se activará una salida de *ERROR*, que me dirigirá a un estado S4. en caso de que *ERROR* sea cero, indica que puede proceder a sumar.

S3: Estado de suma. Solamente frena cuando *FIN* sea igual a uno, indicando que se llegó al valor final y yendo a un estado S5.

S4: Estado de error de carga de datos o de *UPDOWN* incoherente, se debe presionar *RECARGAR* para volver a S0.

S5: Estado donde mi sistema evalúa mediante la entrada *CONTINUAR* si el sistema reinicia el sumador empezando nuevamente o se va al estado inicial S0 a esperar nuevos valores de carga.

S6 y S7: Estados incoherentes o no válidos. No forman parte del flujo normal del sistema. Ante la detección de uno de estos estados, el sistema retorna automáticamente al estado **S0** como medida de seguridad y consistencia operativa.

4. Funciones de Salida

A continuación se describen las señales de salida principales:

- $D_0 = Q_0^+$

$$D_0 = \overline{REC} \cdot [(S_0 \cdot LI) + (S_1 \cdot \overline{LF}) + (S_2 \cdot \overline{ER} \cdot ST) + S_3 + (S_5 \cdot CTN)]$$

- $D_1 = Q_1^+$

$$D_1 = \overline{REC} \cdot [(S_1 \cdot LF) + (S_2 \cdot \overline{ER}) + (S_3 \cdot \overline{FN}) + (S_5 \cdot CTN)]$$

- $D_2 = Q_2^+$

$$D_2 = \overline{REC} \cdot [(S_2 \cdot ER) + (S_3 \cdot FN) + S_4]$$

Siendo:

REC = RECARGAR

LI = LOAD_I

LF = LOAD_F

ER = ERROR

ST = START

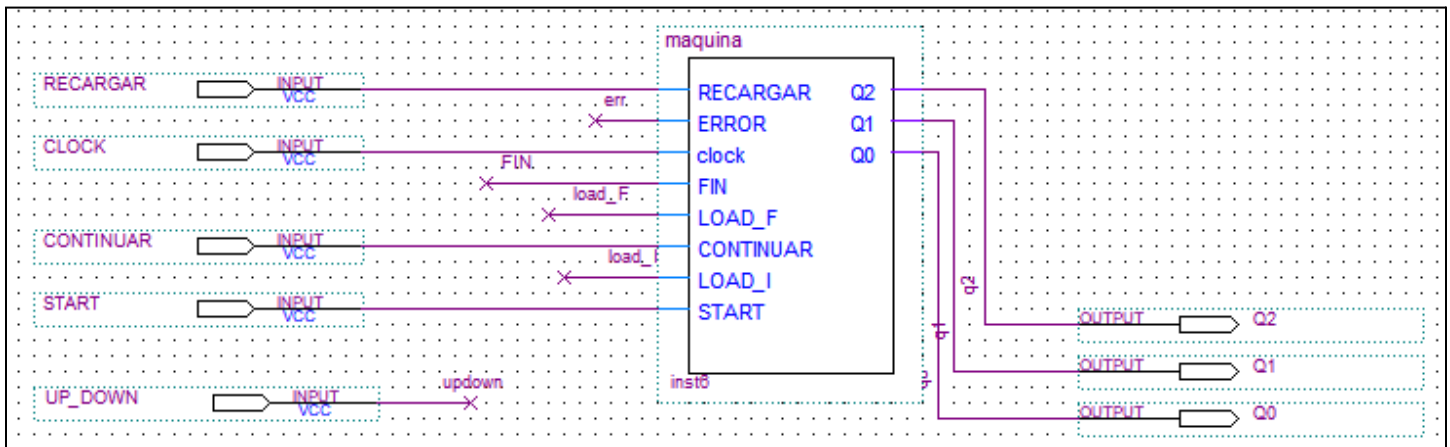
CTN = CONTINUAR

FN = FIN

5. Implementación en Quartus

5.1. Máquina de Estados

Mediante las funciones lógicas obtenidas para la máquina de estados creamos el componente de máquina que transiciona todos los estados de nuestro circuito:



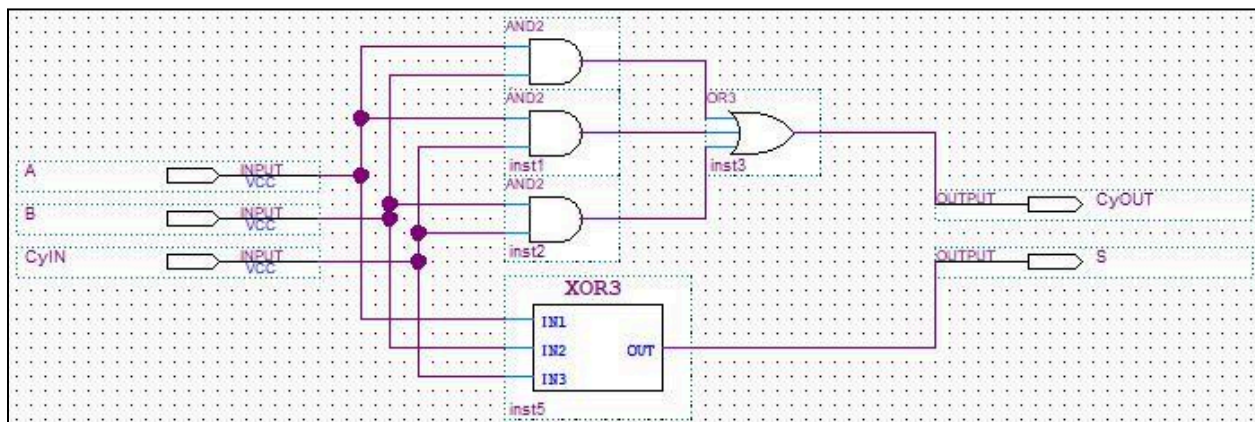
5.2. Sumador de 12 bits con Signo

Para llegar a crear el sumador de 12 bits con signo comenzamos por:

5.2.1. Módulo sumadorCompleto

El módulo sumadorCompleto implementa un sumador binario completo de 1 bit, encargado de procesar dos operandos (A y B) junto con un bit de acarreo de entrada (CyIN). Su salida consiste en el bit de suma (S) y el correspondiente acarreo de salida (CyOUT), respetando las reglas de la aritmética binaria.

Este módulo constituye la unidad elemental sobre la cual se construyen sumadores de mayor ancho.

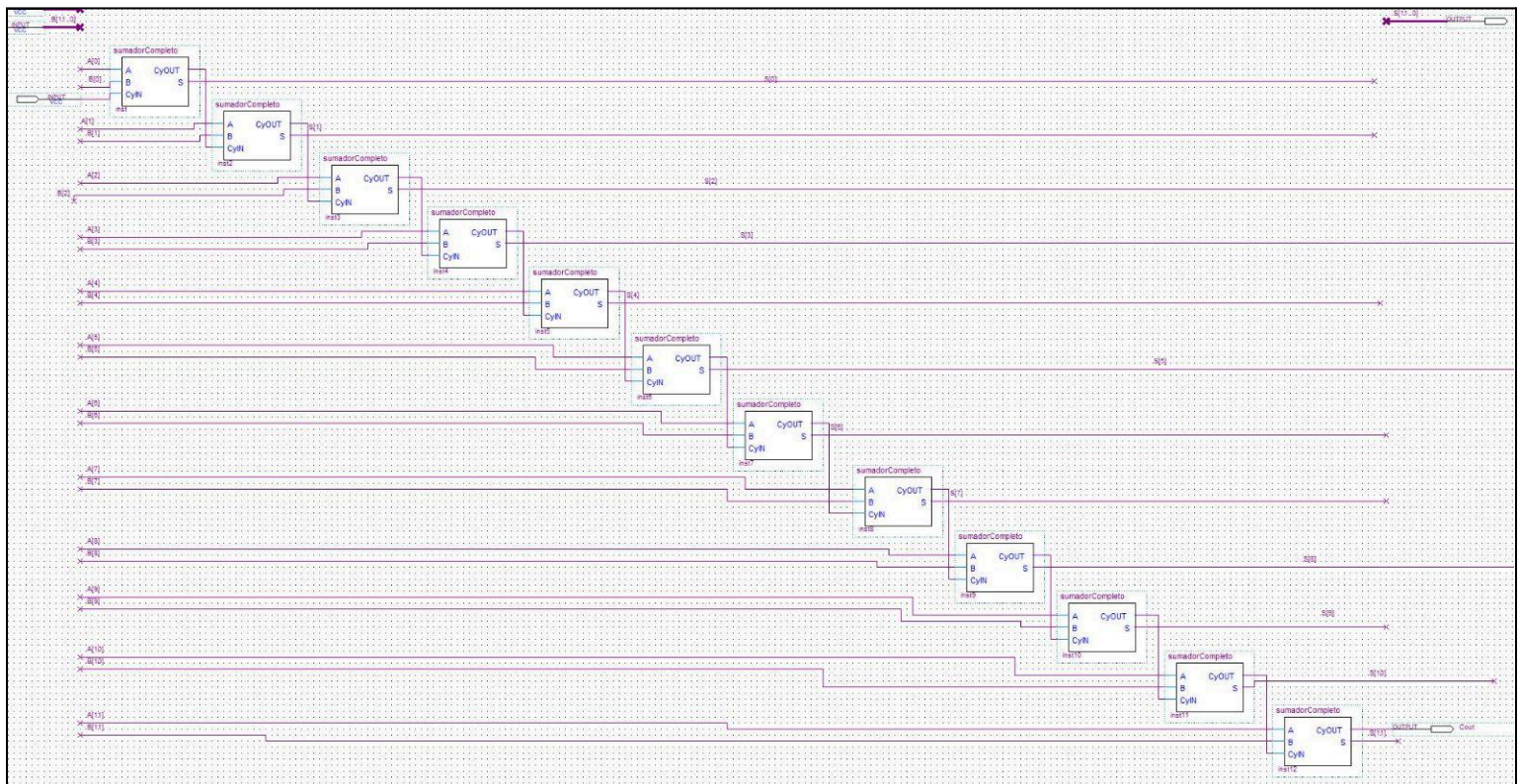


5.2.2. Módulo Sumador de 12 bits (sumador12Bits)

El módulo sumador12Bits tiene como función realizar la suma de dos operandos de 12 bits, incorporando una señal de acarreo de entrada (Cin) y generando como resultado una salida de 12 bits (S) y un acarreo de salida (Cout).

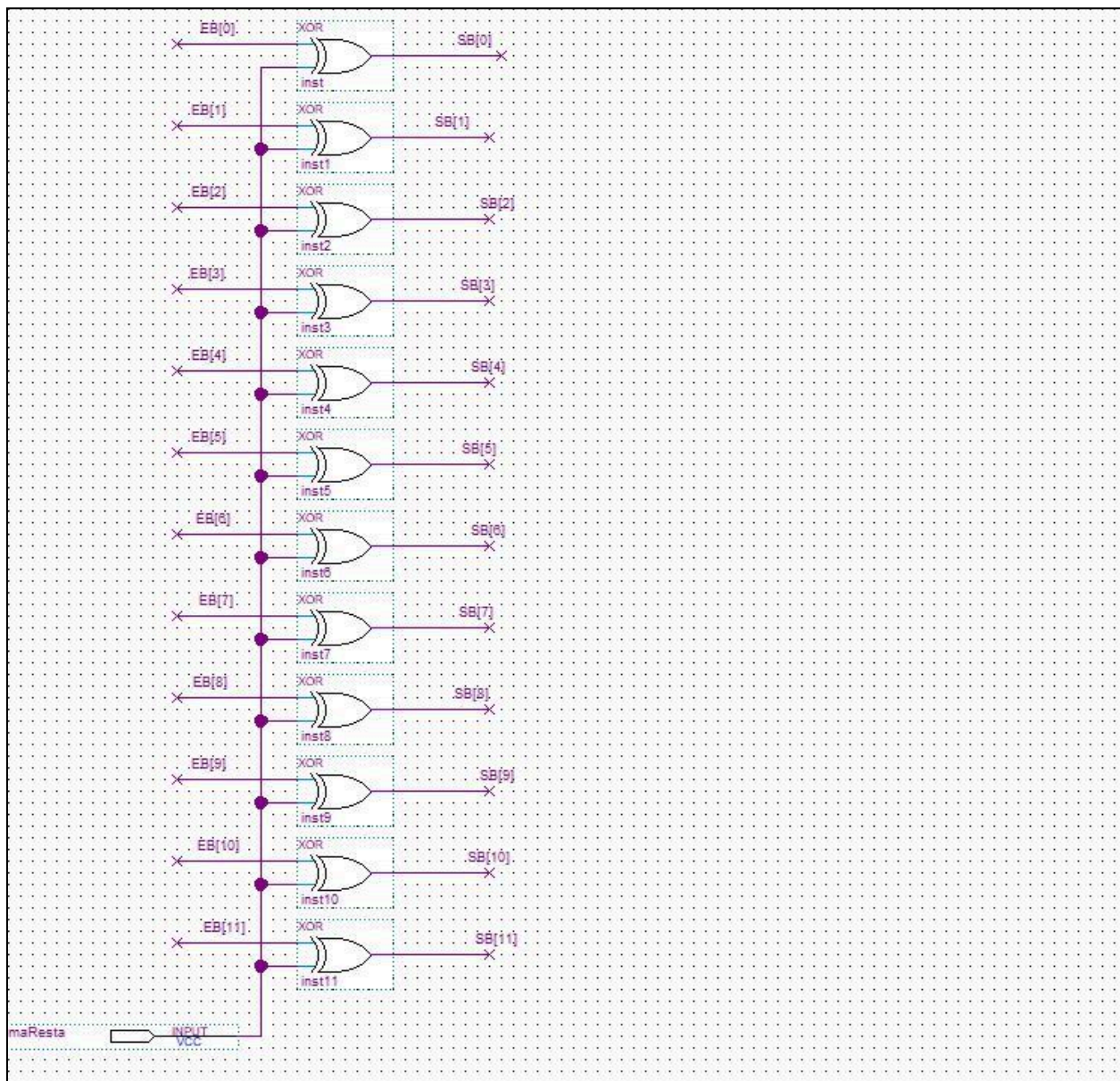
Cada instancia del módulo sumadorCompleto se encarga de sumar un bit de los operandos $A[i]$ y $B[i]$, arrastrando el acarreo de la suma de los bits menos significativos a la suma de los bits mas significativos.

Este sumador se forma con sumadores completos en cascada.



5.2.3. Módulo Inversor

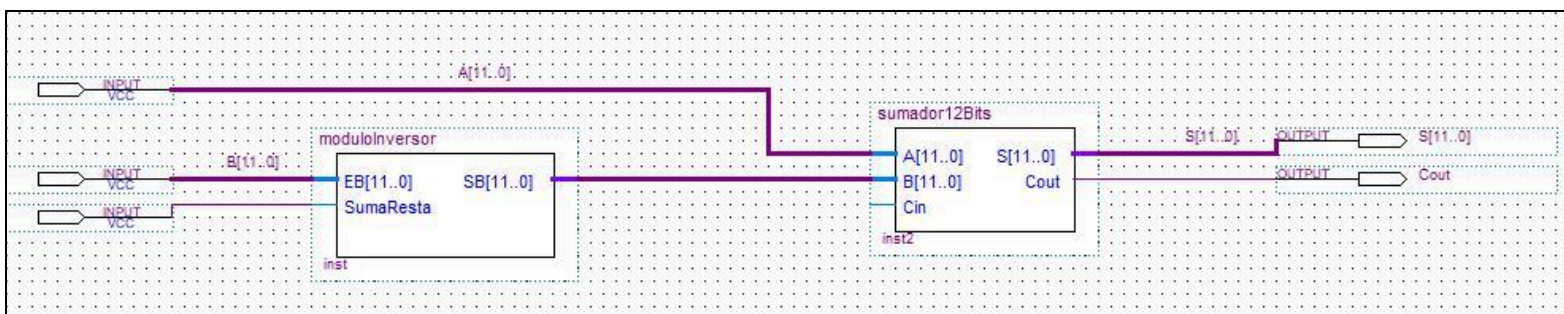
Este módulo fue desarrollado con el propósito de invertir bit a bit una palabra de 12 bits, permitiendo así implementar el complemento a uno como paso previo en operaciones aritméticas sobre números con signo. Su utilización resulta fundamental para luego poder implementar el sumador con signo. La inversión se logra mediante la utilización de compuertas XOR que combinan cada bit de la entrada con una señal de control denominada sumaResta. Cuando esta señal se encuentra activa, el módulo produce como salida el complemento a uno de la entrada; en caso contrario, la salida coincide con la entrada original.



5.2.4. Módulo sumador12BitsPosYneg

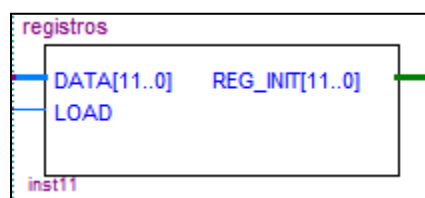
Una vez creados los módulos anteriores, desarrollamos este módulo que permite realizar tanto sumas como restas de números de 12 bits en función de una señal de control externa.

Cuando la señal de control SumaResta se encuentra en nivel lógico bajo, el módulo actúa como un sumador convencional. En cambio, cuando $\text{SumaResta} = 1$, se realiza una operación de resta mediante la técnica de complemento a dos.

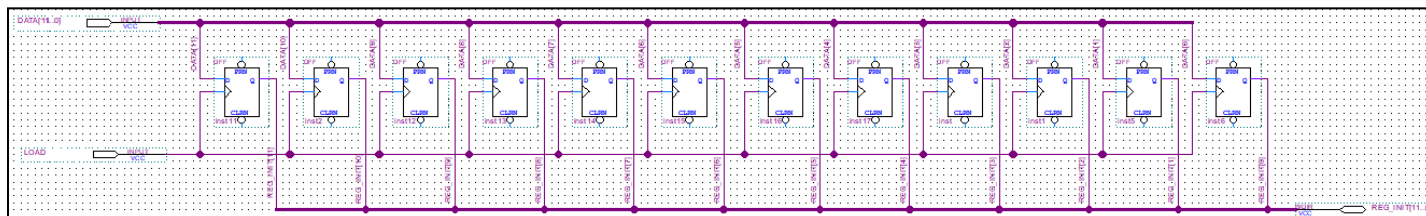


5.3. Registros

Tanto para almacenar el valor inicial como el valor final, utilizamos un registro de 12 bits.



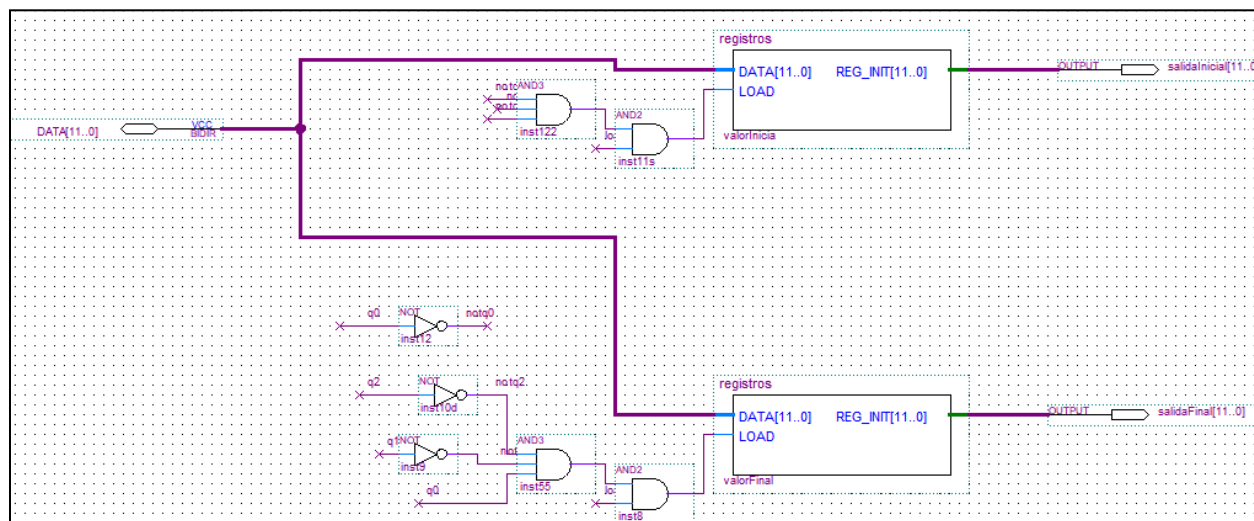
Internamente está diseñado con 12 FF tipo D. Tanto la entrada como la salida son en paralelo:



5.4. Bus DATA[11..0]

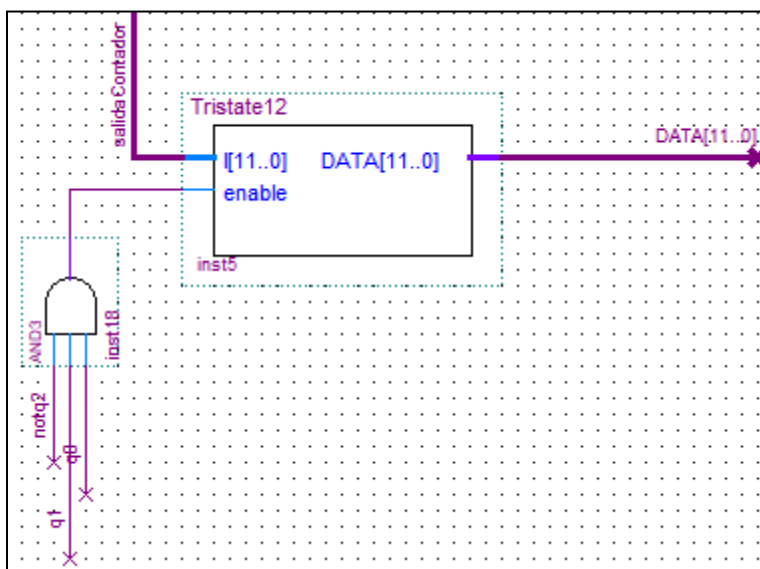
Representamos al bus DATA[11..0] como un bus bidireccional (BIDIR en Quartus). De esta forma se puede usar como entrada y salida, dependiendo el momento de ejecución del programa.

Para la **carga de los valores inicial y final**, conectamos directamente el bus DATA a cada registro inicial y final, ya que estos registros se manejan con su respectiva señal de LOAD. Entonces, nos aseguramos que cada registro tome el valor de DATA cuando se active su señal de LOAD, siempre que nos encontremos en el estado indicado.



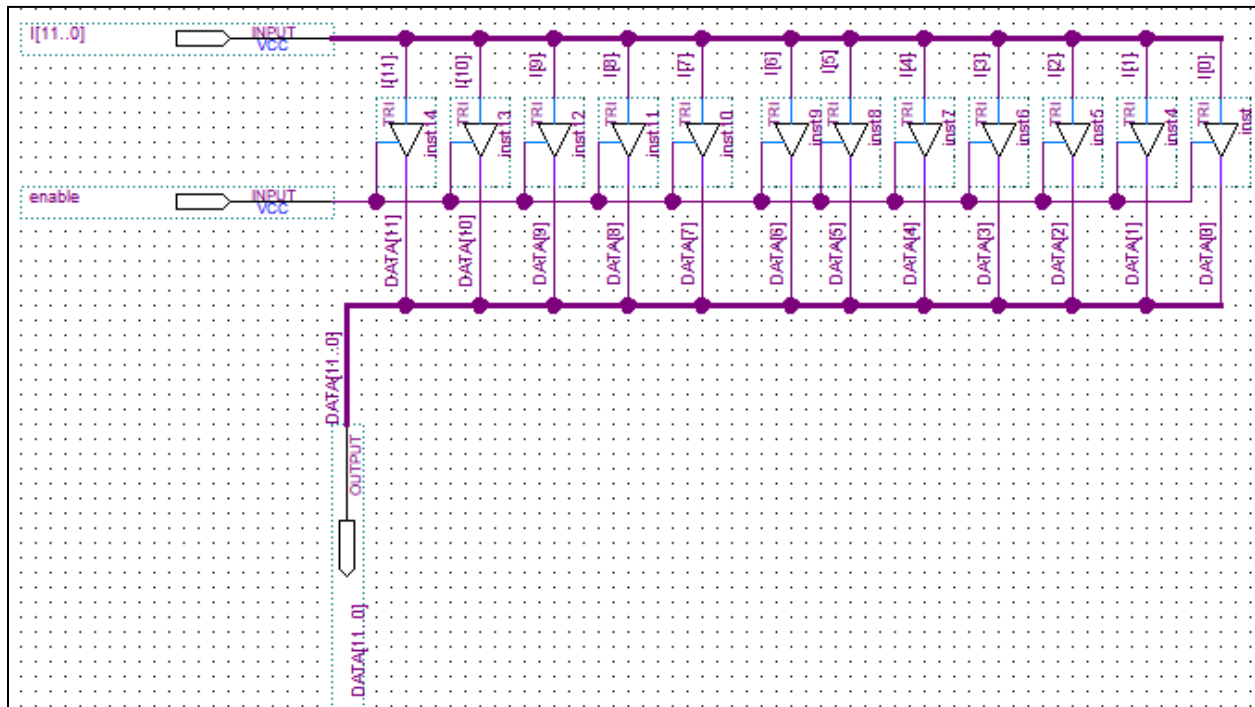
Por otro lado, para manejar la **salida** del valor actual que lleva el contador en el bus DATA[11..0], utilizamos un Tristate de 12 bits.

El Tristate funciona como una llave que deja o no pasar la señal a la salida, que en este caso es el bus DATA[11..0].



Si enable=1, la salida del contador (valor actual de la cuenta) se mostrará en el bus DATA. Configuramos que enable=1 cuando la máquina de estados se encuentra en el estado 011, que es el estado correspondiente a la ejecución del contador.

Internamente, el tristate de 12 bits fue generado con 12 tristates de 1 solo bit, que tienen en común la entrada enable.

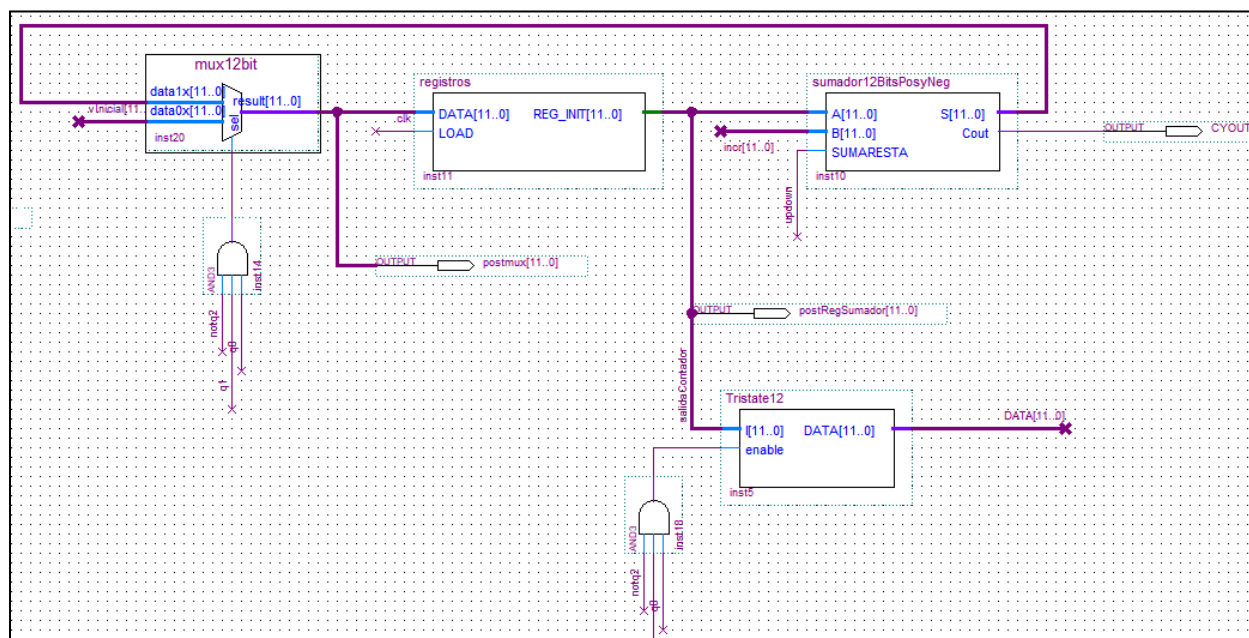


5.5. Contador

La implementación del contador se puede dividir en 3 etapas. La primer etapa contiene un multiplexor de buses. Mientras la señal de control sea 0, deja pasar el valor inicial, que es el valor con el que comienza la cuenta. Cuando la señal de control se vuelve 1 (ocurre en el estado 011 de la máquina de estados), el multiplexor deja pasar el valor del bus que proviene de la retroalimentación.

En la segunda etapa, tenemos un registro que aguarda el valor proveniente del multiplexor y lo deja pasar en el próximo ciclo de reloj.

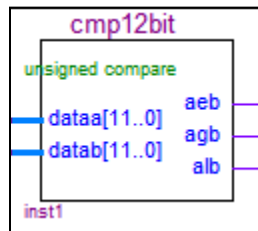
En la tercer etapa, ingresan el valor proveniente del multiplexor y el valor incremento al sumador de 12 bits. Este resultado es llevado mediante la retroalimentación a la entrada nuevamente para continuar el proceso de sumas sucesivas.



Este procedimiento se ejecutará continuamente hasta que se active flag FIN, que cambia el estado de la máquina.

5.6. Implementación del Flag FIN

La base de esta implementación fue el uso de un comparador interno de Quartus:

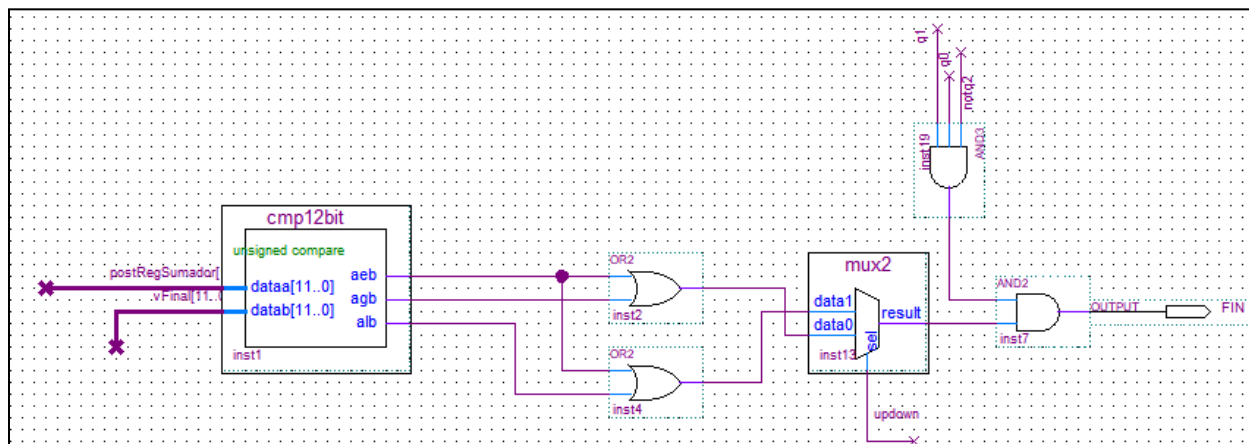


Este tiene como entradas, dos números de 12 bits y tres salidas:

- **aeb**: se activa en alto si dataa=datab
- **agb**: se activa en alto si dataa>datab
- **alb**: se activa en alto si dataa<datab

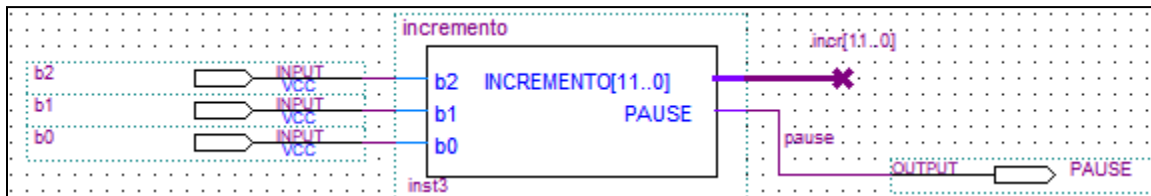
Aprovechando este comparador implementamos la lógica para que, dependiendo el valor de UP/DOWN, es decir, dependiendo si la cuenta es ascendente o descendente, se active la salida FIN al finalizar la cuenta.

Esto ocurre cuando el valor actual del contador es mayor o igual al valor final, en una cuenta ascendente y cuando el valor actual del contador es menor o igual al valor final, en una cuenta descendente. Además, para que FIN se active, la máquina debe estar en el estado 011, que es el estado donde funciona el contador.

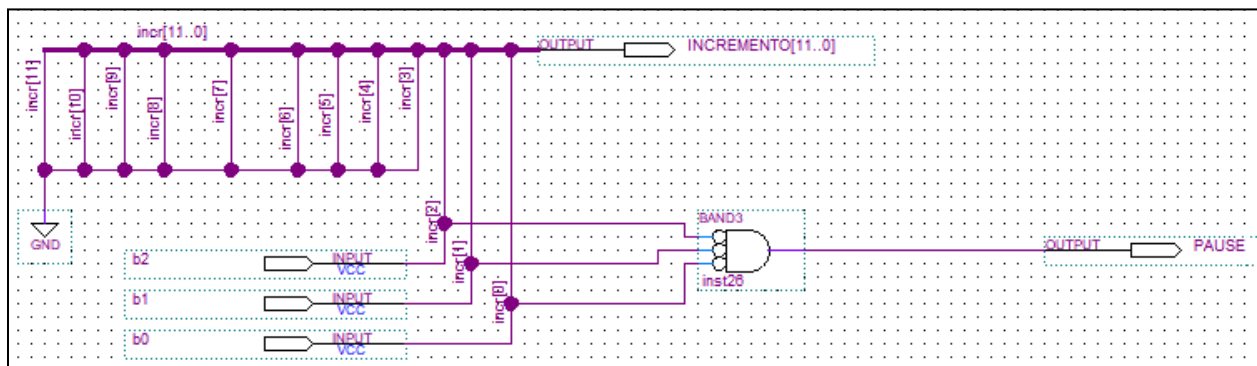


5.7. Implementación del Flag PAUSA

El flag PAUSA debe encenderse siempre que el valor de incremento sea igual a cero.



Componente incremento:



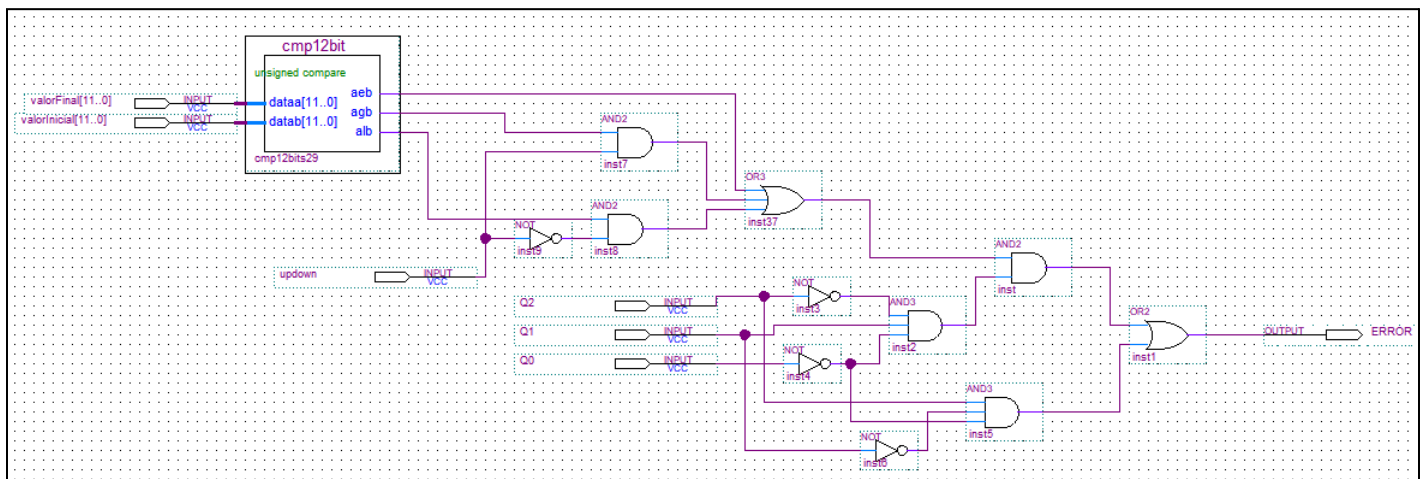
Aprovechamos este módulo para convertir el número incremento de 3 bits en un número de 12 bits. Para eso, tomamos los 3 bits del valor del incremento y completando con ceros lo convertimos en un número de 12 bits.

El tener el incremento como número de 12 bits nos facilita posteriormente la suma de números de 12 bits en el contador.

5.8. Implementación del Flag ERROR

Utilizando el mismo comparador mencionado anteriormente, se enciende la salida ERROR en 1 cuando:

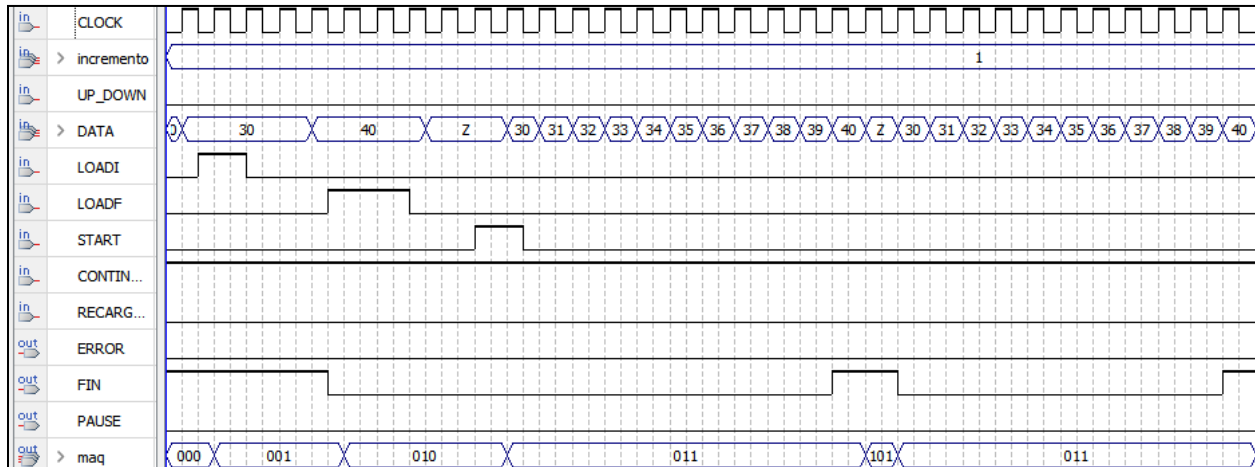
- Estando en UP/DOWN=0 (cuenta ascendente), valorFinal sea menor a valorInicial y además la máquina se encuentre en estado 010.
- Estando en UP/DOWN=1 (cuenta descendente), valorFinal sea mayor a valorInicial y además la máquina se encuentre en estado 010.
- Cuando valorInicial sea igual a valorFinal (sin importar el valor de UP/DOWN) y además la máquina se encuentre en estado 010.
- Cuando la máquina se encuentre en estado 100 (estado de ERROR).



6. Simulación Funcional con Herramienta WaveForm

6.1. Caso 1:

- Contador ascendente (UP/DOWN=0) desde 30 hasta 40, con incremento 1.
- CONTINUAR=1, RECARGAR=0.



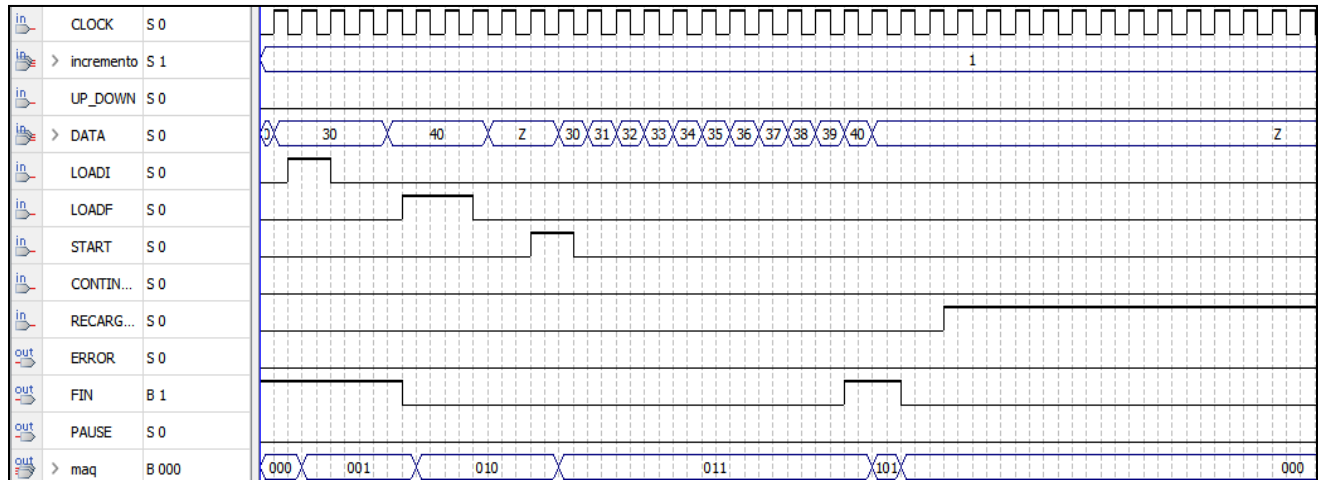
Inicialmente se configuran los valores inicial y final, y mediante las señales LOADI y LOADF se cargan en los respectivos registros internos inicial y final. Luego, mediante la señal START, comienza el conteo en el bus DATA.

Se observa que cuando finaliza la cuenta, al ser CONTINUAR=1, vuelve a comenzar. También, el flag FIN se enciende cada vez que se llega al valor final 40.

Se puede ver la transición de estados de la máquina de estados. Comenzando con 000 (esperando carga de valor inicial), luego 001 (esperando carga de valor final), continúa con 010 (esperando señal START) y luego transiciona entre los estados 011 (estado de sumas sucesivas) y estado 101 (estado en el que finalizo la suma).

6.2. Caso 2:

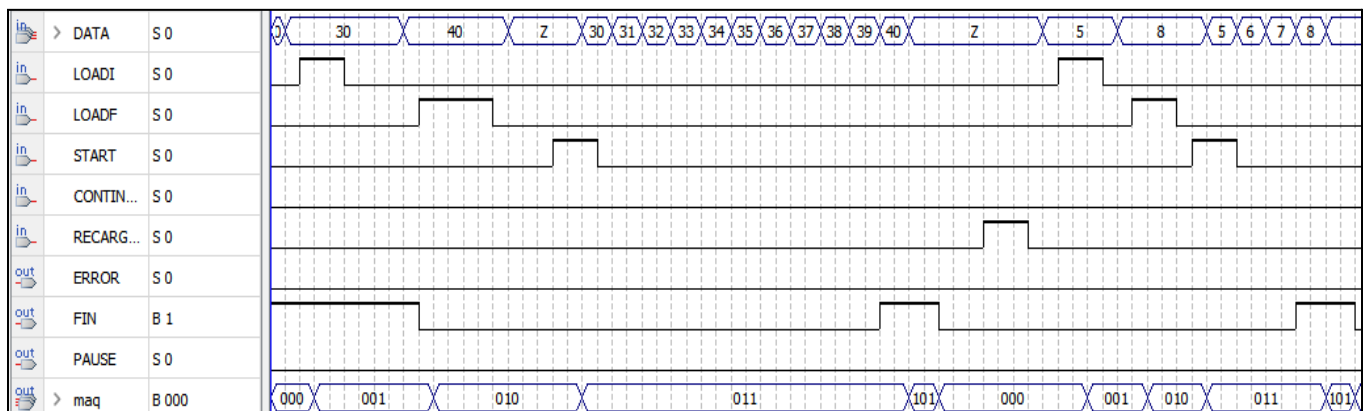
- Contador ascendente (UP/DOWN=0) desde 30 hasta 40, con incremento 1.
- CONTINUAR=0, RECARGAR=1.



A diferencia del caso anterior, CONTINUAR=0, por lo que cuando finaliza el contador, la cuenta se detiene.

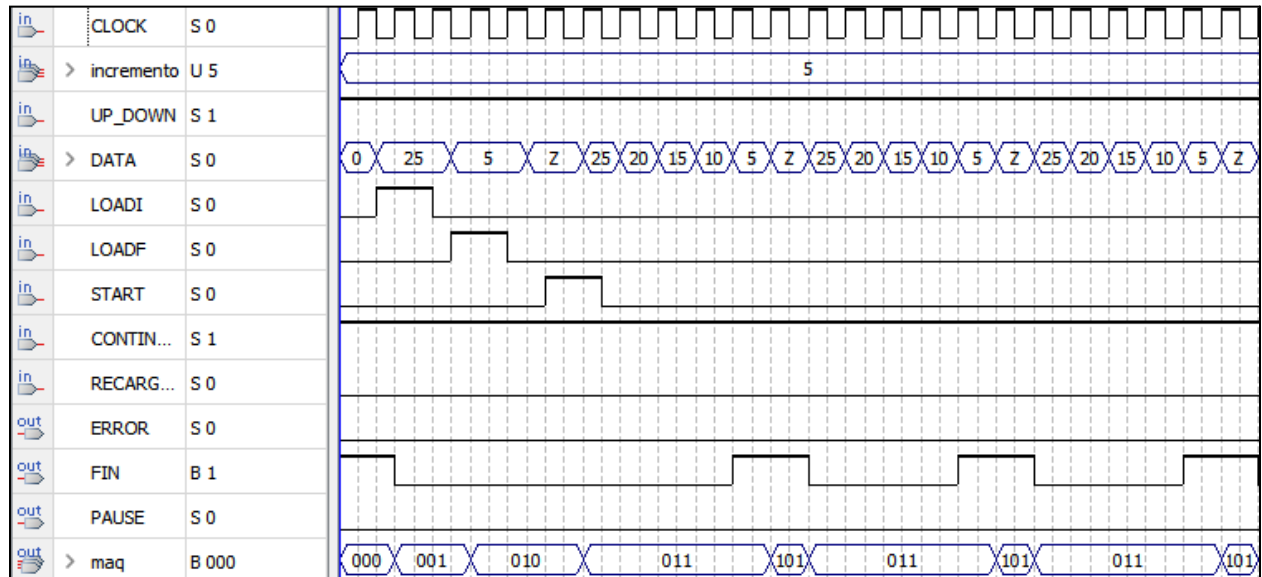
Además, cuando finaliza la cuenta RECARGAR=1, es decir, que el bus espera nuevamente que se carguen valores inicial y final. Esto se puede ver en la máquina de estados, que vuelve al estado inicial 000.

En este mismo ejemplo, probamos ingresar nuevos valores inicial y final y que comience otro conteo:



6.3. Caso 3:

- Contador descendente (UP/DOWN=1) desde 25 hasta 5, con incremento 5.
- CONTINUAR=1, RECARGAR=0.

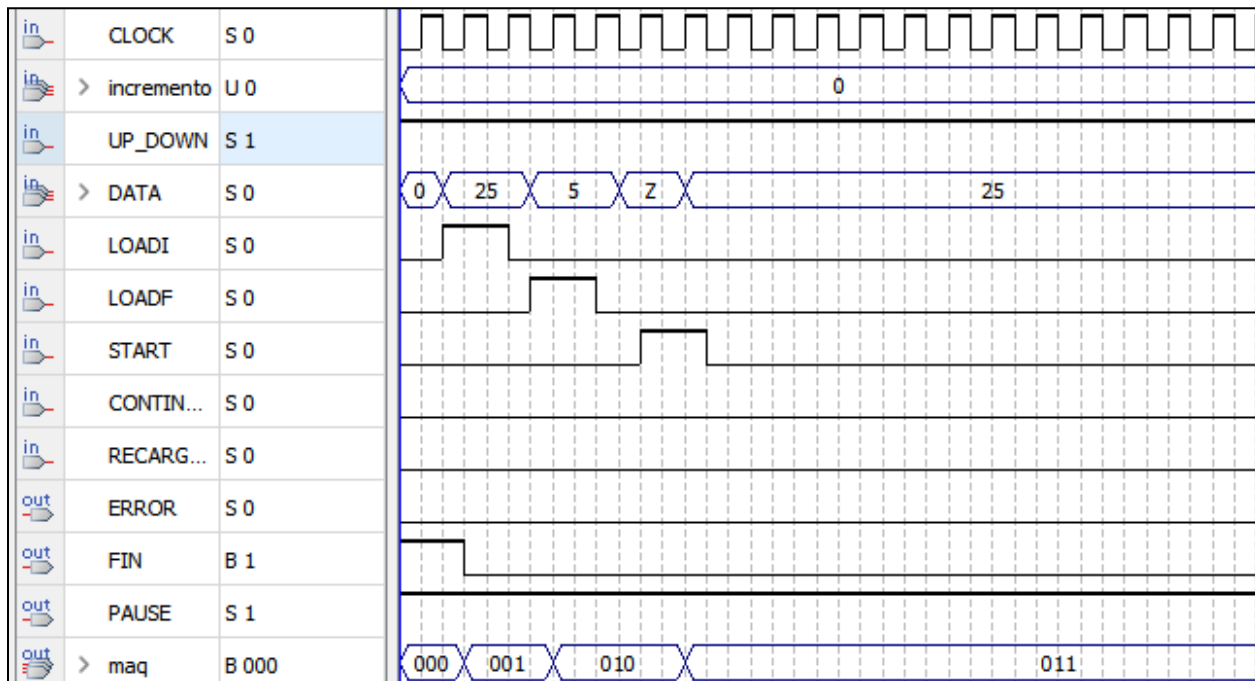


Al igual que casos anteriores, primero se carga el 25, luego el 5, y con la señal START comienza el contador. Al ser CONTINUAR=1, el contador se reinicia al finalizar cada cuenta.

6.4. Caso 4:

Se testea el uso del flag PAUSA

- Contador descendente (UP/DOWN=1) desde 25 hasta 5, con incremento 0.
- CONTINUAR=0, RECARGAR=0.



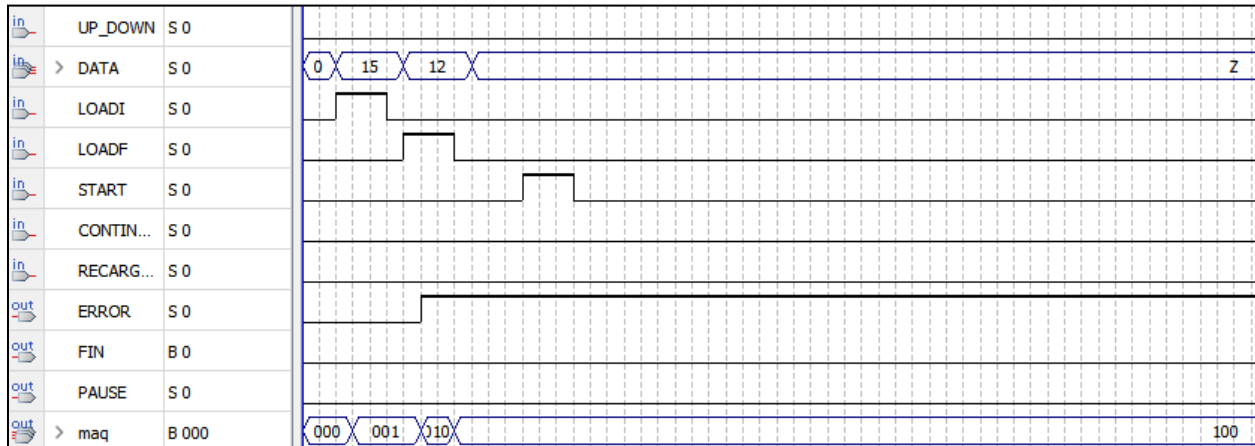
El flag PAUSA debe encenderse siempre que incremento=0.

Notar que, tal como lo diseñamos, la máquina de estados se mantiene en el estado 3, y el bus DATA muestra el valor inicial. Es decir, que es como si se estuviera realizando las restas sucesivas, pero con el incremento en 0.

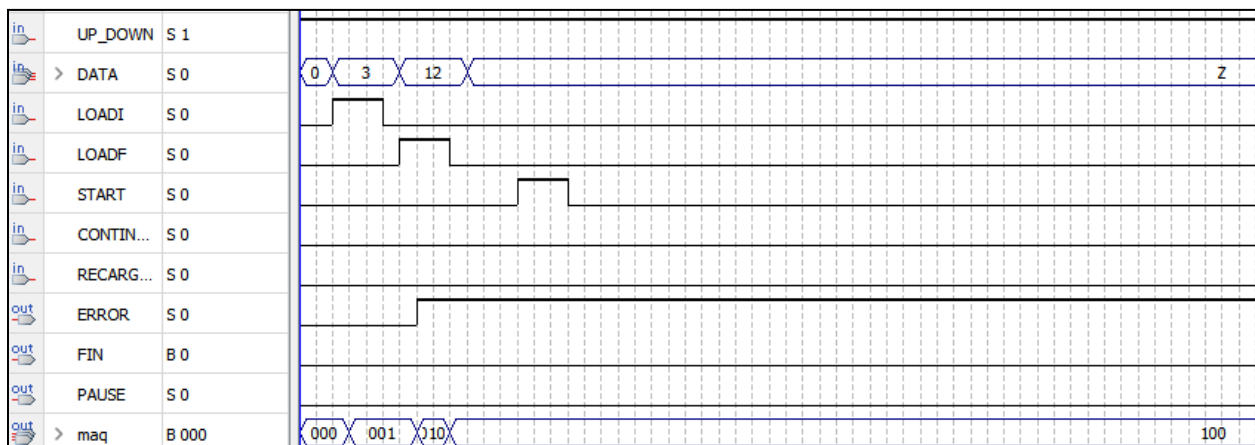
6.5. Caso 5:

Se testea el uso del flag ERROR.

- UP/DOWN=0 (contador ascendente) y valor inicial (15) es mayor al valor final (12).



- UP/DOWN=1 (contador descendente) y valor inicial (3) es menor al valor final (12).



Notar que cuando ERROR=1, la máquina automáticamente va hacia el estado 100 a la espera de que se recarguen los valores.